Tecnicatura Superior en Desarrollo de Software

Módulo 07b — Capa InterfaceAdapter: `ViewModel` y `View (MVVM)`

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

Prerequisitos de lectura

Antes de avanzar con este módulo, se recomienda haber leído:

Módulo 07 — Capa InterfaceAdapter: Presenters y ViewModels
Cubre: el Presenter, `BasePresenter<T>`, cómo se registra en el contenedor de DI. Este módulo retoma esa misma clase y construye lo que faltaba arriba de ella.

Anexo 07e — Fundamentos de Blazor
Cubre: estructura de un componente `.razor`, `@bind`, `@onclick`, `[Parameter]`, y la diferencia entre `@inject` y `OwningComponentBase<T>` para resolver servicios con un scope de DI propio de la pantalla.

Este módulo integra esas dos bases para completar la capa **InterfaceAdapter** del lado de la presentación: el **ViewModel**, que orquesta la llamada al caso de uso y expone el resultado en forma bindable, y la **View**, el componente Blazor que consume ese ViewModel.

¿Qué es un ViewModel?

Un **ViewModel** es el objeto que una View consume directamente: expone el estado de la pantalla como propiedades simples (texto, números, booleanos) y los comandos que el usuario puede disparar como métodos. La View no sabe nada de `UseCase`, de `InputPort` ni de `OperationResult<T>` — solo conoce al ViewModel, hace binding contra sus propiedades y llama a sus

métodos. Es la mitad de MVVM que mira hacia adentro, hacia la capa Application; el Presenter, que ya conocés del Módulo 07, es la otra pieza — la que recibe la notificación cruda del UseCase a través del OutputPort.

¿Por qué hacen falta las dos si ya existe el Presenter? Porque tienen audiencias distintas. El Presenter existe para que el UseCase tenga a quién notificarle el resultado sin conocer nada de UI — su forma está determinada por el `OutputPort`, que a su vez está determinado por lo que el UseCase necesita comunicar (éxito, documento duplicado, error de validación). El ViewModel existe para que la View tenga contra qué hacer binding — su forma está determinada por lo que el formulario necesita mostrar (un texto para el título, un mensaje de error, un booleano para deshabilitar el botón mientras la operación está en curso). Son dos contratos que cambian por razones distintas: el primero cambia si cambia el caso de uso, el segundo cambia si cambia el diseño de la pantalla.

ViewModel	El objeto que la View bindea — orquesta, no decide	InterfaceAdapter

Recibe por constructor el InputPort del caso de uso y el Presenter que va a acumular el resultado. Expone propiedades públicas simples para el binding y un método por cada acción que la View puede disparar. Después de ejecutar el UseCase, lee el resultado del Presenter y traduce eso a sus propias propiedades — la View nunca toca al Presenter ni al UseCase directamente.

Responsabilidades:

- Exponer estado bindable (propiedades públicas simples)
- Construir el Request DTO a partir de ese estado
- Invocar al UseCase a través del InputPort
- Leer el Presenter y traducir el resultado a estado de UI

Analogía del mundo real

Retomando la analogía del intérprete simultáneo del Módulo 07: el Presenter es el intérprete que escucha al orador (el UseCase) y anota la traducción en una libreta. El ViewModel es el **asistente de sala** que toma esa libreta, la resume en un cartel legible desde lejos y decide qué luces prender — verde si salió bien, roja con el motivo si algo falló. El público (la View) no necesita entender la libreta del intérprete ni saber que existe — solo mira el cartel del asistente.

Si mañana la sala cambia de diseño — un cartel más grande, luces en vez de texto — el asistente cambia su forma de resumir, pero el intérprete sigue anotando exactamente igual. Esa independencia es la misma que separa al ViewModel del Presenter en el código.

Características esenciales

Un ViewModel bien formado tiene un perfil reconocible: recibe sus dependencias por constructor (el InputPort y el Presenter), expone solo tipos simples en sus propiedades públicas (nunca DTOs de Application ni el `OperationResult<T>` crudo), y cada método público corresponde a una acción que la View puede disparar directamente desde un `@onclick` o similar.

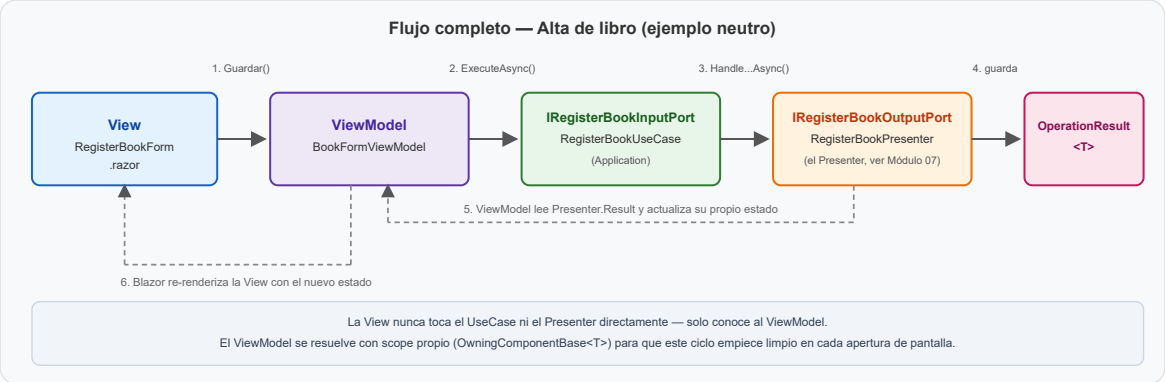
> **¿Por qué no hace falta `INotifyPropertyChanged`?** En WPF con binding XAML nativo, el ViewModel necesita notificar cada cambio de propiedad para que la UI se refresque. En Blazor el mecanismo es otro: cuando un `@onclick` dispara un método y ese método termina, Blazor vuelve a renderizar el componente automáticamente — no hace falta ningún protocolo de notificación mientras el cambio de estado ocurra *dentro* de un manejador de evento de la propia View. Por eso el `BookFormViewModel` de este módulo expone propiedades públicas simples, sin ninguna infraestructura de notificación.

Otra característica clave, ya anticipada en el anexo de Blazor: el ViewModel se resuelve con un scope de DI propio de la pantalla, no del scope ambiente. Eso es lo que le permite guardar estado de una operación puntual (el texto que el usuario

está escribiendo, si hay una operación en curso) sin que ese estado se filtre a la próxima vez que se abra la misma pantalla.

Cómo encaja en la arquitectura: el flujo completo

El flujo arranca en la View, pasa por el ViewModel, atraviesa el InputPort hacia el UseCase, y vuelve por el OutputPort hacia el Presenter — que el ViewModel lee para actualizar su propio estado y, con eso, la View.



Notá que este flujo no agrega ninguna capa nueva de dependencia: el ViewModel vive en el proyecto InterfaceAdapter, igual que el Presenter. La View, por ser un componente Razor, vive en un proyecto aparte de la misma capa (se explica más abajo). La única pieza realmente nueva respecto del Módulo 07 es el ViewModel — la View existía conceptualmente desde antes, pero recién ahora tiene contra qué hacer binding.

Marco teórico complementario

El ejemplo completo: `BookFormViewModel`

```
1  public sealed class BookFormViewModel
2  {
3      private readonly IRegisterBookInputPort _useCase;
4      private readonly RegisterBookPresenter _presenter; // el Presenter, del Módul
   o 07
5
6      // propiedades bindables — tipos simples, nada de DTOs de Application acá
```

```
    public string Titulo { get; set; } = string.Empty;
    public string Isbn { get; set; } = string.Empty;
    public string Mensaje { get; private set; } = string.Empty;
    public bool EnCurso { get; private set; }

    // el contenedor de DI inyecta ambas dependencias en el mismo scope
    public BookFormViewModel(
        IRegisterBookInputPort useCase,
        RegisterBookPresenter presenter)
    {
        _useCase = useCase;
        _presenter = presenter;
    }

    // el comando que la View dispara desde el botón — método público, sin paráme
tros
    public async Task GuardarAsync()
    {
        EnCurso = true;

        try
        {
            // construye el Request DTO a partir de su propio estado bindable
            var request = new RegisterBookRequest(Titulo, Isbn);
            await _useCase.ExecuteAsync(request);

            // lee el Presenter — nunca al UseCase directamente — y traduce a est
ado de UI
            Mensaje = _presenter.Result!.IsSuccess
                ? $"Libro '{Titulo}' registrado."
                : string.Join("; ", _presenter.Result.Errors);
        }
        finally
        {
            // se libera siempre, aunque el caso de uso falle: el botón nunca que
da deshabilitado
            EnCurso = false;
        }
    }
}
```

La View: `RegisterBookForm.razor`

```
@inherits OwningComponentBase<BookFormViewModel>

<h3>Alta de libro</h3>

```

```
5  <input @bind="Service.Titulo" placeholder="Título" />
6  <input @bind="Service.Isbn" placeholder="ISBN" />
7
8  <button @onclick="Service.GuardarAsync" disabled="@Service.EnCurso">
9      Guardar
10 </button>
11
12 <p>@Service.Mensaje</p>
```

Todo el binding apunta a `Service` — la propiedad que `OwningComponentBase<BookFormViewModel>` resuelve desde el scope propio de esta pantalla. No hay ningún `@inject` suelto en este componente: es la aplicación directa de lo que viste en el Anexo 07e.

Dónde vive la View: un proyecto aparte

Una View es un archivo `.razor`, y un proyecto común de C# **no sabe compilarlo**: lo ignora en silencio. La compilación termina con cero errores y cero advertencias, pero la View nunca existió. Por eso las Views se alojan en un proyecto especial, y la dependencia entre proyectos apunta en un solo sentido: **Views referencia a InterfaceAdapter**, nunca al revés.

¿Por qué las Views van en una biblioteca de clases de Razor?

Podría pensarse que alcanza con agregar los archivos `.razor` a un proyecto que ya existe. Técnicamente se puede (cambiando su SDK y agregando el paquete de Blazor), pero separarlas en su propio proyecto, una **biblioteca de clases de Razor** (*Razor Class Library*), tiene cuatro razones concretas:

1. **Es lo que sabe compilar un `.razor`.** Un proyecto común de C# ignora los `.razor` sin avisar: termina con 0 errores y la View no existe. La plantilla de biblioteca de clases de Razor viene con el SDK correcto, el paquete de Blazor y el archivo de importaciones ya configurados, así que no queda ningún paso manual que se pueda olvidar.
2. **Mantiene limpia la arquitectura.** El proyecto `LibraryApp.InterfaceAdapter` (Presenters y ViewModels) queda en C# puro, sin depender de Blazor. Se puede probar, y reutilizar, sin ninguna tecnología de pantalla. Si mañana la interfaz cambiara de tecnología, solo se reemplaza el proyecto de Views.

3. **Es una pieza pensada para ser referenciada.** Una biblioteca de clases de Razor es el formato estándar para compartir componentes: el host (la aplicación de escritorio con Blazor Hybrid) simplemente la referencia y muestra sus pantallas. Además puede llevar consigo recursos estáticos (hojas de estilo, scripts, imágenes en `wwwroot`) que viajan con la biblioteca.
4. **Deja los límites visibles en la solución.** La dirección de la dependencia queda escrita en una referencia de proyecto: Views referencia a InterfaceAdapter, y nunca al revés.

La contrapartida es honesta: hay **un proyecto más** en la solución. Se paga con un archivo de proyecto adicional y una referencia, y a cambio cada proyecto tiene una sola responsabilidad.

El archivo de proyecto de las Views declara tres cosas, todas explícitas:

```
<Project Sdk="Microsoft.NET.Sdk.Razor">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="9.0.20" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\LibraryApp.InterfaceAdapter\LibraryApp.InterfaceAdapter.csproj" />
  </ItemGroup>

</Project>
```

- `Microsoft.NET.Sdk.Razor` — el kit de compilación que entiende los `.razor`.
- **El paquete** `Microsoft.AspNetCore.Components.Web` — los tipos de Blazor (`OwningComponentBase`, `@onclick`, `@bind`…), con versión exacta de la misma rama que el framework del proyecto.
- **La referencia al proyecto InterfaceAdapter** — para poder usar los ViewModels.

Y el archivo de importaciones, que en este ejemplo contiene:

```
@using Microsoft.AspNetCore.Components.Web
@using LibraryApp.InterfaceAdapter.Books.ViewModels
```

El error que no avisa: el primer `@using` es el que da vida a los eventos. Si falta, la View **compila sin ningún error** pero `@bind` y `@onclick` no hacen nada: el botón se ve y no responde. Un error de compilación se corrige en cinco minutos; uno silencioso se descubre en producción. Por eso se prueba la View renderizándola (actividad práctica, Parte E).

_Imports.razor: cómo funciona y cómo usarlo bien

En un archivo `.cs` cada archivo empieza con sus `using`. Un componente Razor también puede llevar sus `@using` arriba, pero en la práctica se repetirían en todas las Views. Para eso existe `_Imports.razor`: un archivo de texto con los `@using` que comparten los componentes. Estas son las reglas para usarlo correctamente:

- **El nombre es exacto:** `_Imports.razor` (guion bajo, *I* mayúscula, plural). Con cualquier otro nombre, el compilador lo trata como un componente más y no lo aplica.
- **Se aplica solo, sin referenciarlo:** no hay que invocarlo desde ninguna View. El compilador de Razor lo detecta y agrega esos `@using` a todos los `.razor` de *su carpeta y de las subcarpetas*.
- **Puede haber más de uno:** uno en la raíz del proyecto para lo común, y otros dentro de una carpeta para lo que solo necesitan sus componentes. Los de la raíz y los de la carpeta se acumulan.
- **Qué va ahí:** la línea de eventos web que trae la plantilla (`@using Microsoft.AspNetCore.Components.Web`) y los namespaces de los tipos que usan tus Views, como los de los ViewModels (`@using LibraryApp.InterfaceAdapter.Books.ViewModels`). **Qué no va:** lógica ni código.
- **La línea de eventos web nunca se borra:** sin ella la View compila, pero `@bind` y `@onclick` no responden.
- **Si cambia un namespace, se actualiza acá:** si movés un ViewModel a otro namespace, hay que corregir la línea correspondiente del `_Imports.razor` (en las Views no hay nada que tocar).

> **Cómo diagnosticar** cuando una View dice que no encuentra un tipo, en este orden: (1) ¿el `@using` del namespace de ese tipo está en el `_Imports.razor`? (2) ¿el proyecto de Views referencia al proyecto que contiene ese tipo? (3) ¿está instalado el paquete NuGet que lo define? Casi siempre es la primera.
>
> Así se ve la diferencia en una View. Sin `_Imports.razor`, cada componente tendría que repetir las líneas de importación; con él, empieza directamente por lo que lo hace único:
>
> **Sin** `_Imports.razor` — cada View repetiría las importaciones:

```
@using Microsoft.AspNetCore.Components.Web
@using LibraryApp.InterfaceAdapter.Books.ViewModels
@inherits OwningComponentBase<BookFormViewModel>

<h3>Alta de libro</h3>
<!-- ...los campos... -->
```

> **Con** `_Imports.razor` — la View empieza directamente por lo suyo:

```
@inherits OwningComponentBase<BookFormViewModel>

<h3>Alta de libro</h3>
<!-- ...los campos... -->
```

Buenas y malas prácticas

La View no debería conocer al UseCase

ERROR COMUN

La View inyecta el InputPort directamente y arma el Request DTO en su propio `@code`. El ViewModel deja de tener sentido — la View terminó haciendo su trabajo.

```
<!-- MAL: la View conoce Application directamente -->
@inject IRegisterBookInputPort UseCase

@code {
    private async Task Guardar()
        => await UseCase.ExecuteAsync(new RegisterBookRequest(titulo, isb
n));
}
```

BUENA PRACTICA

La View solo conoce al ViewModel. Armar el Request, invocar el UseCase y leer el Presenter es responsabilidad exclusiva de `BookFormViewModel`.

```
<!-- BIEN: la View solo conoce al ViewModel -->
@inherits OwningComponentBase<BookFormViewModel>

<button @onclick="Service.GuardarAsync">Guardar</button>
```

El ViewModel no debería validar reglas de negocio

ERROR COMUN

El ViewModel decide si el ISBN es válido antes de llamar al UseCase. Esa regla ya vive en el Value Object del dominio — ahora hay dos lugares que pueden desincronizarse.

```
// MAL: regla de negocio duplicada en el ViewModel
if (Isbn.Length != 13)
    Mensaje = "ISBN inválido";
```

BUENA PRACTICA

El ViewModel siempre llama al UseCase y confía en que el dominio (Value Objects, Guards) rechace lo inválido. Solo traduce el resultado que el Presenter ya acumuló.

```
// BIEN: la validación vive en el dominio, el ViewModel solo traduce
await _useCase.ExecuteAsync(request);
Mensaje = _presenter.Result!.IsSuccess ? "OK" : ...;
```